

	制定日期	2025/09/20			
	修改日期				
测试开发工程师工作指导书					
拟制部门	核准	关联部门	制定人	版本	总页 数
测试中心	陈颖		颜顺鹏	V1.0	20 页

变更记录

[illegible]

测试开发工程师工作指导书

1.前言

1.1 编写目的

本指导书的目的是为了规范项目测试工具/脚本的开发流程，提高自动化测试用例的执行效率，确保测试用例能够高效、稳定地执行。通过本手册的指导，开发人员能够按照统一的规范进行测试工具的开发，并能及时将测试工具与自动化测试用例关联起来。

1.2 重要性

- 提升测试效率，减少人工操作
- 保证脚本可维护性
- 提升产品质量和稳定性

2.岗位基本信息

岗位属性：

- 岗位名称：测试开发工程师（TSE）
- 岗位性质：全职
- 工作地点：公司办公
- 协作部门：测试团队、BMC 研发团队、BIOS 研发团队、硬件研发团队、产品管理部

技术栈要求：

- 编程语言：Python、Shell、Java、JavaScript、NSIS 等脚本语言
- 测试框架：Unittest、Selenium、Pytest、PySide6、Appium、JUnit、TestNG 等
- 版本控制工具：Git、SVN 等
- 持续集成工具：Jenkins、GitLab 等
- 数据库：MySQL、Oracle 等（根据项目需求）
- 操作系统：Linux、Windows 等

软技能要求：

- 良好的沟通能力，能够与项目团队及其他相关部门有效协作
- 强烈的责任感，能够按时交付高质量的代码
- 团队合作精神，能够与团队成员共享技术和经验
- 问题分析能力，能够快速定位并解决技术问题

3.文档适用范围

适用对象：

- 自动化测试开发人员
- 测试工具维护人员

- 相关技术人员（开发、运维等）

适用场景：

- DML 系统上新功能或更新功能的测试
- 新增或维护自动化测试脚本
- 测试工具/脚本的开发和维护

4.术语和缩写

- DML：测试用例管理平台（Data Management Library）
- CI/CD：持续集成/持续部署（Continuous Integration/Continuous Deployment）
- Git：分布式版本控制系统
- SVN：集中式版本控制系统
- Unittest：自动化测试框架
- Jenkins：开源自动化服务器，用于持续集成
- RDM：需求驱动管理平台（Require Driver Management）

5.岗位方向分类及定义

测试开发工程师（TSE）根据技术专长和业务需求分为以下七个核心方向：

1. 需求确认/承接
 - 核心目标：确认测试需求转化自动化的可行性，按照规则计划承接转化任务。
2. 项目测试工具开发/维护
 - 核心目标：将重复性高、业务稳定的测试需求转化为自动化脚本，提升保障测试效率。
3. 测试开发技术研究
 - 核心目标：探索新技术并推动落地，解决团队技术瓶颈。
4. 项目自动化测试执行
 - 核心目标：通过自动化手段执行测试任务，输出高质量测试报告。
5. 持续集成测试工具开发/维护
 - 核心目标：构建稳定的 CI/CD 测试流水线，确保代码持续交付质量。
6. 运维管理工具开发、维护
 - 核心目标：通过开发和维护运维管理工具，简化日常运维流程，自动化常见的运维任务，从而提高运维效率，降低人工干预的频率。

6.各方向工作职责与流程

6.1 需求确认/承接

6.1.1 职责范围

确认在 RDM 上指派的需求开发，确保 RDM 上需求完整实现，减少手动测试的重复性工作，提高测试效率和准确性。测试工具/脚本开发的职责范围主要包括：

1. 需求确认

工具开发需求来源：团队内部仅存在一个 RDM 系统

其中涉及对外发布（运维工具，送测工具等给到客户现场），必须 PMM 上产品经理进行提交需求，PMM 届时会自动流转到 RDM 中

- 测试工具需求且 DML 存在用例待转自动（包含测试套件，压力测试等）：判断 DML 系统的待转自动化用例，是否具备可转自动化条件。
- 测试工具需求 DML 上不存在用例：评估 RDM 上提出的需求，是否能够转换自动化，并明确没有的用例的原因进行注明，除运维工具送测工具此类不涉及 BMC BIOS HW 性能类的测试工具，原则上必须具备 DML 用例。
- 运维工具类：明确首次需求 PMM 存在工具开发任务，并跟需求提出人明确细节，客户需求等

2. 需求创建与承接

- 根据需求沟通结果，由需求方或自行在 RDM 平台进行任务创建（DML 存在用例的，需需求上注明工具号）。
- 承接人员选择上可以直接指派开发人员或与上级主管沟通指定开发人员。

3. 需求开发与流转

- 在任务开发周期内开始任务和更新状态。
- 任务完成后及时更新状态并流转给需求提出方。

6.1.2 工作内容

1. 需求确认

- **DML 用例确认：**根据 DML 系统的待转自动化用例，确认各步骤能否转化。遇到不能转化的情况和用例编写人员进行沟通修改步骤，修改后仍不满足转化条件的拒绝该用例转化需求，满足转化条件的用例可进行需求创建。
- **压力脚本需求确认：**和需求提出人员沟通压力脚本的预处理步骤、循环逻辑、检查点等，确认沟通无误且可实现后进行需求创建。
- **RDM 上需求逐步核对，关键在于测试点和需要检查点是否完整**

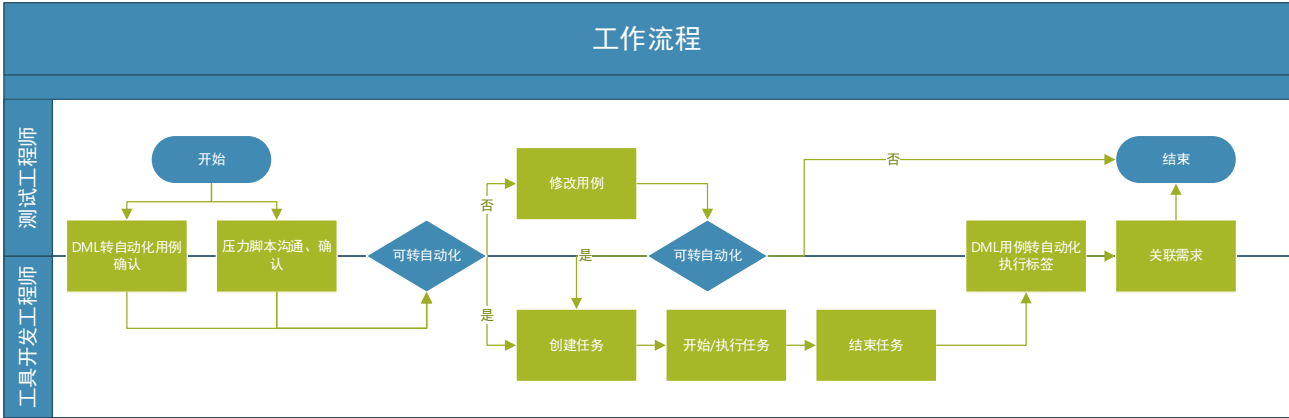
2. 需求创建与承接

- **需求创建：**DML 用例转化任务一般由测试开发工程师自行安排计划，按照每天或每周可转换用例数量创建任务，任务需注明转化用例名称和用例号。压力脚本任务一般由需求提出人进行创建，任务需进行简要描述、明确使用项目、机型等信息。
- **需求承接：**需求任务创建时可以指定开发人员，自行创建的任务一般指派给自己，需求提出人员可以和上级主管沟通指定开发人员。当工时紧张，不能及时处理相关任务时，可以和上级主管沟通，将任务转派给其他测试开发工程师进行处理。

3. 需求开发与流转

- **需求开发：**承接任务后进行排期，填写预估开发周期和工时，在预估周期内及时开始任务，填写任务所属项目、仓库信息。任务开始后按照规范进行开发（项目测试工具开发/维护章节会做详细说明，本章不展开）。
- **需求流转：**已开始的任務，每天在 RDM 平台上更新进度。完成开发和自测的任务，要及时点完成，RDM 平台会自动推送开发完成的通知到任务创建人。DML 用例转化自动化完成后，在 DML 平台将该用例转化为自动化执行并关联需求。

6.1.3 工作流程



6.2 项目测试工具开发/维护

6.2.1 职责范围

- 测试脚本的开发与管理，确保测试工具正常执行，减少手动测试的重复性工作，提高测试效率和准确性。产品或测试用例的持续开发和更新时，测试脚本需要根据新的功能需求进行维护和优化，确保自动化测试能够及时反映新需求并稳定执行。

1. 工具命名与版本控制

- 按照命名规范给脚本进行命名。
- 按照版本控制规范进行版本管理。

2. 脚本开发与编码规范

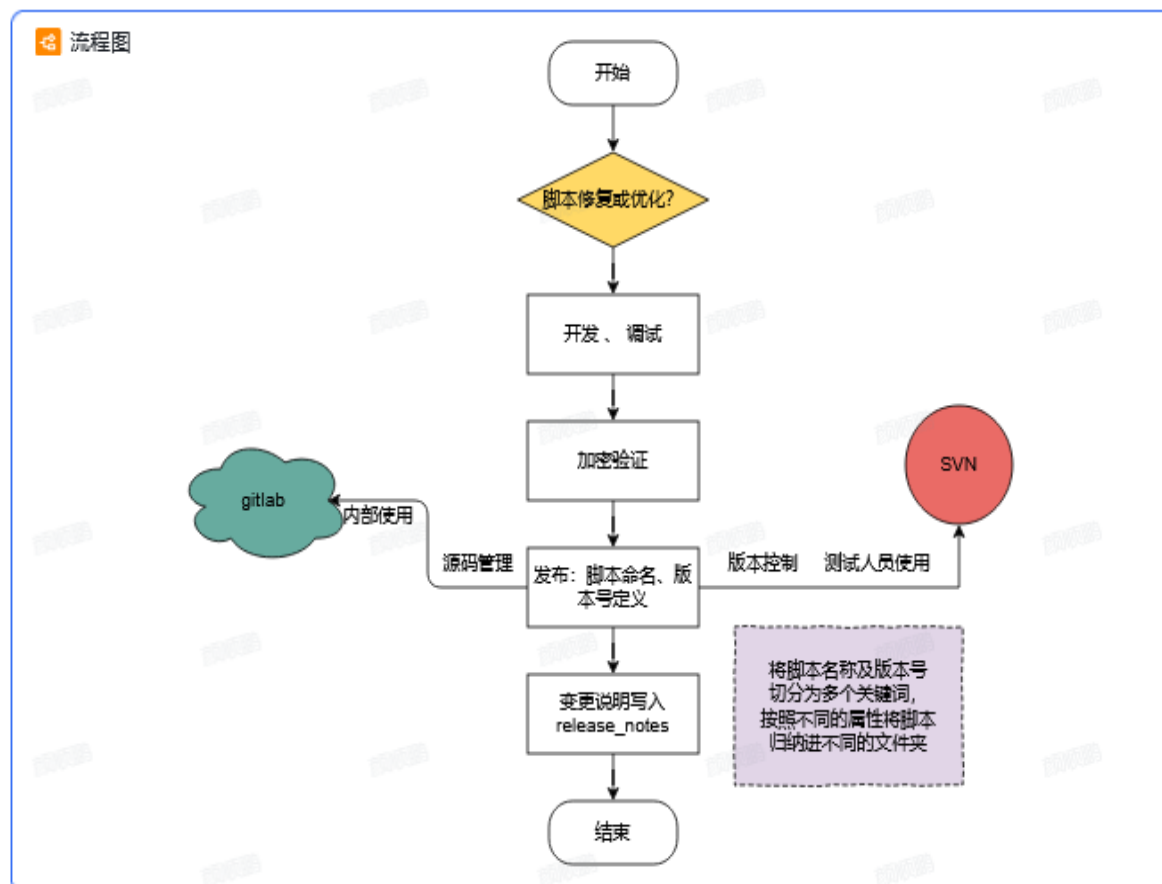
- 根据测试需求，分析待开发的测试用例并将其转化为自动化脚本。
- 确保每个自动化脚本都能够准确执行测试用例，覆盖测试点。
- 开发自动化测试脚本时遵循统一的编码规范，包括命名规范、代码结构等。
- 使用版本管理工具（如 Git 和 SVN）提交代码，确保代码可追溯与维护。
- 编写完善的脚本注释和文档，便于后期维护和其他开发人员理解。

3. 脚本更新与维护

- 根据日常测试反馈修正存在的问题，提升脚本稳定性。
- 根据 DML 系统的功能更新，持续更新和优化测试脚本。
- 编写更新日志（Release Notes），明确版本更新内容。
- 提交代码至 Git，更新可执行代码至 SVN。

4.脚本发布与存放

- BVT 脚本更新，进行 Git 提交。
- 压力脚本更新，进行 Git 提交、打包、上传 SVN。



5.支持与培训

- 对测试团队成员进行工具和脚本使用培训，提供技术支持和指导，提升团队整体技术水平。
- 为所开发和维护的测试工具和脚本编写详细的使用文档，确保团队成员能够快速上手。

6.2.2 工作内容

1. 工具命名与版本控制

- 工具命名应采用英文关键字拼接，重点体现出关键对象（如 BIOS、BMC、PSU、Fru 等）、关键功能（如 Update、Switch 等）、测试模式（如 IPMI、Redfish 等），避免与已有工具命名重复或近似的情况
- 版本控制分为小版本和分支版本管理，小版本号。分支版本按照机型进行划分，目前工具版本管理从 G30 开始按位标识代表该分支支持对应机型，如 3450，代表该分支支持 G30、G40、G50 机型测试；如 0006，代表该分支支持 G60 机型测试；AMD 机型分支在分支号后添加大写“A”进行标识，如 0006A，代表该分支支持 G60 AMD 机型测试。

2、命名格式规范：

字段1 _ 字段2 _ 字段3 _ 字段4 _ 字段5

3、字段说明：

字段	说明
字段1	脚本所属的模块（全大写）：BMC or BIOS or Hardware or Performance or VAT or Platform
字段2	脚本性质：Stress or Suite or Other；VAT 下是OMT,AfterSales,Debug,Other(公共库，加密工具都属于增值工具的范围)
字段3	脚本功能描述(除专有名词外首字母大写)，eg:单用户登陆压力：SingleUserLogin(驼峰命名)；OMT 下暂分为Utool UpdateTool BatchTool Automation 是字段3内容)
字段4	使用项目：Universal（通用版）JDM 版本，Customize 以上三个大类，如果有小类则是Universal-Nanhai JDM-Qiandaohu 这样
字段5	版本号（参考第二部分内容）

4、实例：

单用户登陆压力命名：

通用版： BMC_Stress_SingleUserLogin_Universal_版本号

Openbmc(目前就是宜春的openbmc版本)： BMC_Stress_SingleUserLogin_Openbmc_版本号

•

1、版本号定义规范：

V 字段1(X) . 字段2(XX) . 字段3(XX) . 字段4(XXX) . 字段5 (X)

2、字段说明

字段	说明
字段1	主版本号，从1开始。除了初版以外，重大调整时修改（eg：服务器版本升级）
字段2	次版本号，从0开始。新增检测点，新增测试功能时修改
字段3	扩展版本号，从0开始。bug修复、测试逻辑优化时修改 如果是运维工具，该字段表示不通项同项目需求变更（增删改功能）时的版本号。
字段4	BMC/BIOS脚本 :支持的服务器型号(4位)。0000-定制化机型；3450-G30&G40&G50；0450-G50&G40；0050-G50；0400-G40；3000-G30，3456-G30 G40 G50 G60 HW脚本（其实就是带内工具） ：支持的OS版本。1--Centos,Redhat,2--Rocky；3-Debian 17 表示 Centos7 同理（centos redhat rocky本身都是一套，区分大版本即可）；虚拟机上市4--虚拟机运行和windows上；0010 表示redhat centos通用型，0018表示就redhat 8系统（如果7 8都支持就写0010） 运维工具 ：也应该需要此字段，自行查看此工具是属于带内还是带外 一些other类工具，不需要指定操作系统也不需要指定G30 G40 G50的，统一为0000
字段5	BMC代码版本 。A版本；B-openbmc版本（只有BMC的相关工具需要该字段） 运维工具 ：也区分A O 售后或者HW工具：P是python 版本，S是shell版本 一些other类工具，进行区分python shell版本

3、实例

BMC：

Openbmc版：BMC_Stress_SingleUserLogin_Openbmc_V1.1.1.0000.O

- 字段1：BMC，该脚本是测BMC的；
- 字段2：Stress，该脚本是测压力的；
- 字段3：SingleUserLogin，该脚本测的功能是单用户登陆；
- 字段4：Openbmc，该脚本是Openbmc项目专用的；
- 字段5：V1.1.1.0000.O
 - 字段1：1，该脚本的第一个正式版；
 - 字段2：1，该脚本发布后新增过一次测试点；
 - 字段3：1，该脚本进行过一次代码优化；
 - 字段4：0000，该脚本只适用定制化机型；
 - 字段5：O，该脚本只适用Openbmc（非ByteBMC的Openbmc）

HW：

通用版：HW_Stress_SAT_Universal_V1.1.0.01

- 字段1：HW，该脚本是测HW的；
- 字段2：Stress，该脚本是测压力的；
- 字段3：SAT，该脚本测的功能是SAT；
- 字段4：Universal，该脚本是适用每一个通用项目；
- 字段5：V1.1.0.01
 - 字段1：1，该脚本的第一个正式版；
 - 字段2：1，该脚本发布后新增过一次测试点；
 - 字段3：0，该脚本尚未进行过代码优化；
 - 字段4：01，该脚本只适用CentOS操作系统。

运维工具：

Bilibili utool：VAT_OMT_Utool_Bilibili_V1.0.4.0400.A（该命名仅用于SVN）

- 字段1：VAT（），改脚本是增值工具；
- 字段2：OMT（Operation and maintenance tool），改脚本是运维工具；（还会有售后工具，debug工具等）
- 字段3：Utool，该脚本是Utool；
- 字段4：Bilibili，该脚本是哔哩哔哩项目的utool；
- 字段5：V1.0.4.0400.A
 - 字段1：1，改脚本是Utool的初版（第一版大框架）；
 - 字段2：0，改脚本尚未做过大框架的改动；
 - 字段3：4，改脚本在客户（bilibili）的要求下进行的第4版更新
 - 字段4：0400 是G40机型
 - 字段5：A AMI 代码版本

售后工具：

AfterSale LogCollect：VAT_AfterSale_LogCollect_Universal_V1.0.0.0010.P（该命名仅用于SVN）

- 字段1：VAT（），改脚本是增值工具；
- 字段2：AfterSale 脚本是售后工具
- 字段3：LogCollect，该脚本是信息收集工具；
- 字段4：Universal，该脚本是通用使用；
- 字段5：V1.0.4.01.P
 - 字段1：1，改脚本是初版（第一版大框架）；
 - 字段2：0，改脚本尚未做过大框架的改动；
 - 字段3：4，改脚本在小版本更新次数
 - 字段4：0010 是Redhat Centos 7 8 9 系统通用机型
 - 字段5：P python代码版本

2.脚本开发与编码规范

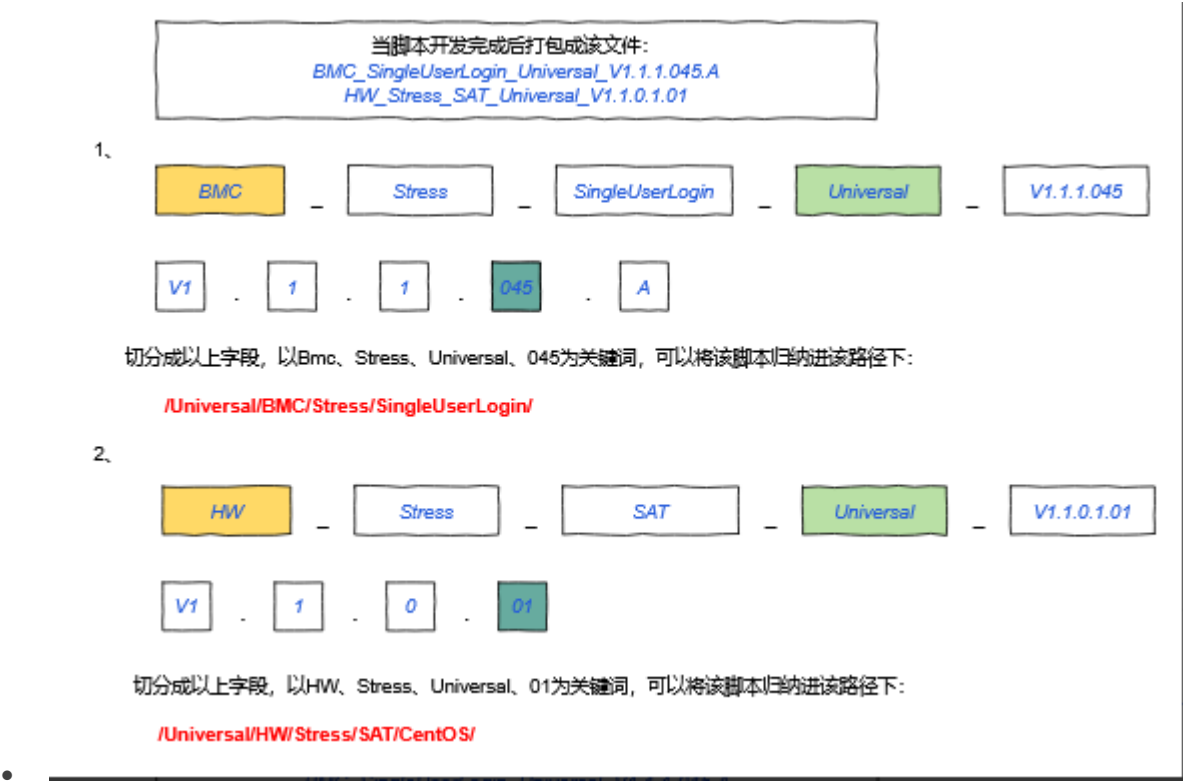
- 通过与开发人员、测试人员密切沟通合作，理解测试需求，在排期内完成功能开发。
- 保证开发完成的脚本逻辑正确，严格按照用例步骤逐一覆盖，没有遗漏测试点，保证测试结果符合预期。
- 严格遵循 Python PEP8 代码规范进行编写，确保代码语法正确，结构清晰。
- 目前团队统一使用 Git 进行提交，提交要保证分支符合版本控制要求，要及时、完整提交，避免遗漏和冲突。
- 脚本内应标注关联的 DML 用例号，压力脚本应同时在 desc.yml 文档内对核心步骤进行简要描述，便于维护和其他人理解。

3.脚本更新与维护

- 日常反馈的问题，及时进行修正、提交、发布流程；测试执行期间遇到的问题提高处理优先级，立即处理立即发布。
- 定期对自动化测试脚本进行回归测试，确保脚本在不同版本之间的兼容性。存在用例更新时，和用例编写人员沟通确认后修正脚本。
- 修正脚本后，如需打包发布需要在 release 文档内新增修正内容描述并更新版本号。
- 修正脚本后，Git 提交时要注明修改内容，有必要可以简要描述修改背景，便于后期追踪。

4.脚本发布与存放

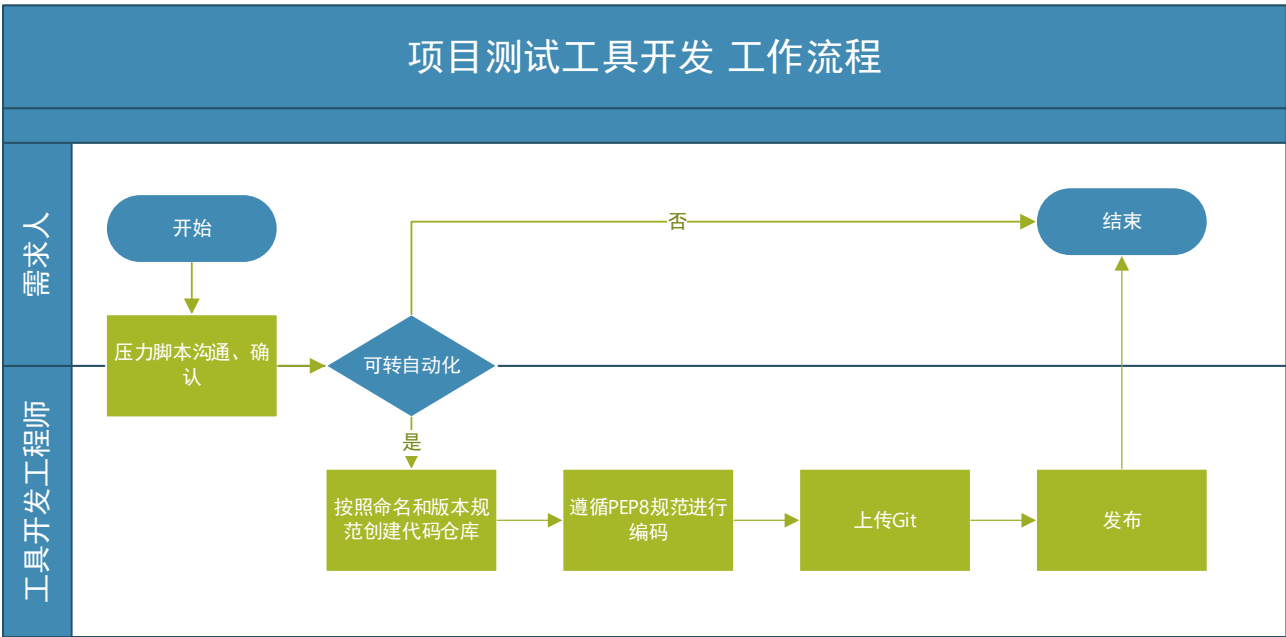
- BVT 套件（Redfish、IPMI、Snmp、Oem）脚本更新后，需要测试开发工程师进行 Git 上传，RDM 平台会固定时间间隔同步新代码，测试人员重新创建新的测试任务即可使用最新代码测试。
- 压力脚本更新后，需要测试开发工程师进行 Git 上传，然后将修改的部分同步上传至 SVN（目录：
http://10.32.129.41/svn/SIT/new_release_tool/BMC_Stress_BMCSuperStress_Universal/BMCSuperStress/）。需要编译发布的工具（多数已合并至 Super 套件，无需进行编译），在编译完成后，将编译文件上传至 SVN 对应目录。测试人员在 SVN 对应目录下下载新上传的工具进行测试。

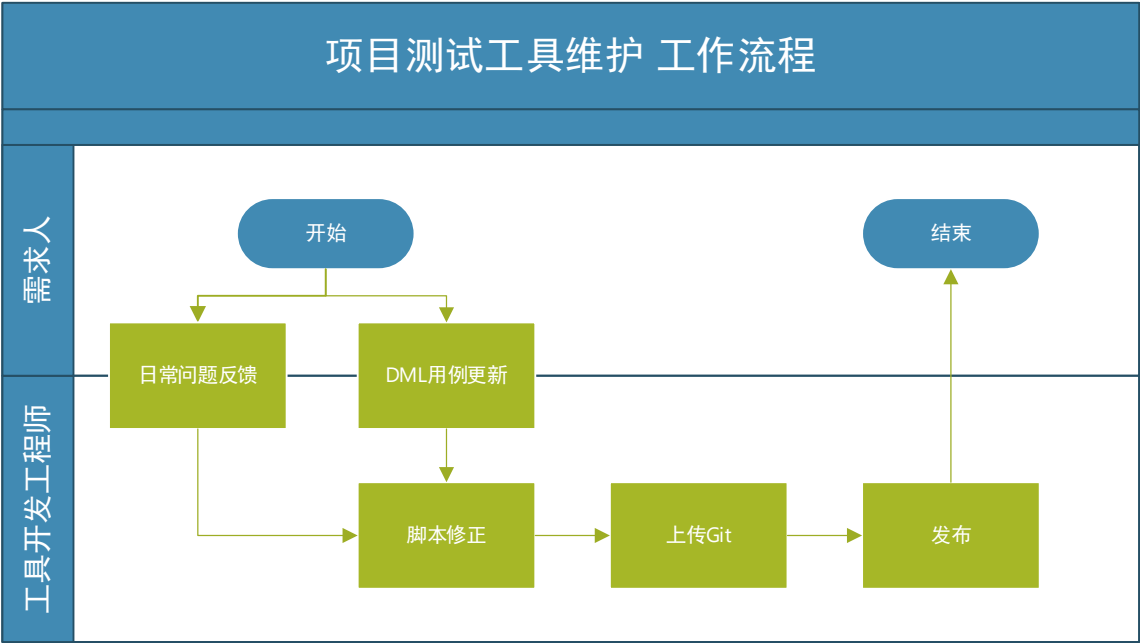


5.支持和培训

- 为团队成员提供技术支持, 帮助他们解决使用测试工具和脚本时遇到的问题。
- 定期为团队举办培训课程, 帮助新成员快速掌握工具和脚本的使用技巧。
- 编写测试工具和脚本的详细文档, 包括使用手册、安装指南、FAQ 等。
- 更新测试工具和脚本的版本发布说明, 确保团队成员了解最新的功能和修复内容。

6.2.3 工作流程





6.3 测试开发技术研究

6.3.1 职责范围

负责相应实验室自动化测试框架的调研和持续改进满足测试需要的各项特性，持续提升测试开发效率。测试工具/脚本开发的职责范围主要包括：

1. 测试框架调研和改进：
- 通过了解项目需求和研究行业内主流测试框架，分析优缺点选择合适的框架或进行二次开发。
 - 定期评估当前使用的框架，提出优化方案，在满足项目需求的同时确保框架工具的高效性和准确性。
2. 提升测试开发效率：
- 优化测试框架，提高易用性和可维护性，提升用例转化效率。
 - 加强团队协作，提升自动化程度，减少人工耗时。
 - 深入学习服务器产品和关注前沿测试技术，提升知识储备，便于在未来工作中引入。

6.3.2 工作内容

1. 测试框架调研和改进：
- 调研：**通过和项目人员沟通，了解项目的具体需求。研究主流测试框架（如 Unittest、Selenium、Appnium 等），评估框架应用和二次开发。定期评估当前的框架在执行效率、运行准确、功能完善等方面是否满足需求。
 - 改进：**评估当前框架识别到瓶颈和不足后，提出优化方案，例如提高执行效率、增强可维护性、减少测试的冗余等。
2. 提升测试开发效率：
- 优化框架：**通过引入模块化设计等方式，简化框架管理和维护工作，提升测试框架的

易用性和可维护性。

- **团队协作：**保持和团队人员的充分沟通，提高测试过程的执行效率、减少流程耗时。
- **个人提升：**在工作中总结和归纳，注重个人学习和积累。

6.3.3 工作流程

1. 测试框架调研和改进：

- 与项目人员沟通，确认自动化需求，选择合适的测试框架实现。
- 收集测试人员的反馈和测试框架的执行结果，完善改进框架。

2. 提升测试开发效率：

- 复盘测试结果持续更新框架，提升转化自动化用例效率。
- 保持和团队人员充分沟通，提高工作流程和测试执行的效率。

6.4 测试项目执行

6.4.1 职责范围

负责项目中自动化测试用例的执行，测试记录填写，Bug 提交验证，测试报告整理。测试项目执行的职责范围主要包括：

1. 测试用例执行和填写

- 执行 DML 平台中测试计划内划分的测试用例。
- 执行测试用例后，在 DML 平台填写测试执行结果。

2. Bug 提交和验证

- 在 Mantis 平台上报 BUG。
- 验证 Mantis 平台指派的 BUG。

3. 测试报告整理

- 自动化生成测试报告。
- 根据自动化生成的报告整理测试结果回填 DML 或在 Mantis 上提交 Bug。

6.4.2 工作内容

1. 测试用例执行和填写

- **执行：**和 TPM 沟通分配的测试计划，已有自动化覆盖的测试项执行相关测试；没有自动化覆盖的进行评估是否能在测试周期内开发完成并执行，不能开发的转派给手动测试，在项目周期内完成测试计划。
- **填写：**执行测试结束后，收集和分析测试结果，将执行结论填写至 DML 平台。

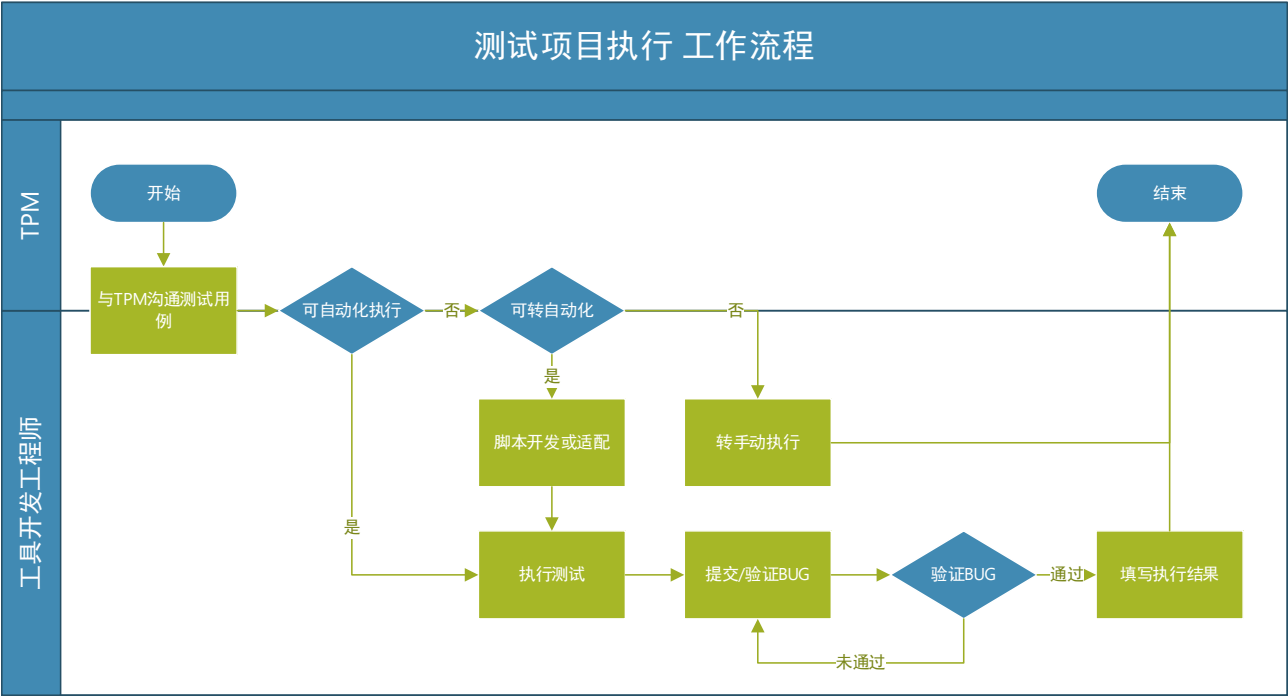
2. Bug 提交和验证

- **提交：**在执行自动化测试过程中发现问题并手动确认后，在 Mantis 平台提交 Bug。
- **验证：**Bug 重新流转到名下后，要及时验证，填写评论和测试结果。

3. 测试报告整理

- 自动生成报告：框架要支持自动生成测试结果的特性。
- 整理：根据自动化报告回填 DML 或在 Mantis 提交 Bug。

6.4.3 工作流程



6.5 持续集成测试工具开发/维护

6.5.1 职责范围



1.工具/脚本开发与维护：

- 负责开发、优化和维护持续集成（CI）测试工具和自动化测试脚本，确保与研发团队密切协作，实时验证项目中已实现的功能。

2.自动化测试与手动验证结合：

- 在测试工具开发过程中，手动验证已经实现的功能，确保能够有效集成自动化测试用例。对于可自动化的测试项，编写并优化自动化测试脚本。

3.CI 流水线支持：

- 支持并优化 Jenkins 等持续集成工具的集成与配置，确保每天能自动化跑 DailyRun 测试。

4.故障分析与反馈：

- 对于 DailyRun 测试失败的项，进行详细的 debug 和问题分析，并形成日报报告。并在每日晨会分享分析结果与相关问题的反馈，并在禅道上报 bug。

5.与研发结对开发：

- 与研发人员进行紧密配合，确保开发过程中的功能符合测试要求，在禅道上和 dailyrun 日报上快速反馈测试结果，及时调整开发进度和功能实现。

6.5.2 工作内容

1.自动化测试脚本的编写与优化

- 根据研发团队开发的功能，快速编写自动化测试脚本并集成到持续集成流水线中。
- 针对功能需求，不断优化现有的自动化测试脚本，提升测试的覆盖率、稳定性和执行效率。
- 编写清晰易读的测试脚本，便于团队成员理解和修改。

2.手动验证与自动化测试结合

- 对于开发完成的功能，手动进行验证，确保开发的功能符合需求并能顺利运行。
- 将可实现自动化的测试用例转化为自动化脚本，进行自动化验证，减少人工干预。

3.持续集成与 DailyRun 的配置与优化

- 配置和优化 Jenkins 等持续集成工具的 CI/CD 流水线，确保测试脚本能够自动化执行并提供实时反馈。
- 每天定时执行 DailyRun 测试，验证开发团队提交的代码是否影响现有功能。
- 维护测试环境的稳定性，确保测试在隔离的环境中进行，减少外部环境因素的干扰。

4.测试结果分析与问题排查

- 对于 Jenkins DailyRun 测试结果进行分析，识别测试失败的项。
- 针对失败的测试项进行详细的 debug，查找问题的根本原因，确保问题能够及时解决。
- 针对失败项，编写调试报告，并推动研发团队进行修复。

5.日报与晨会分享

- 每天根据 DailyRun 的测试结果，生成日报，记录所有失败测试项的 fail 原因。
- 在晨会上分享测试结果和问题反馈，讨论未解决的问题以及可能的优化方案。
- 跟踪并反馈上一轮测试中发现的问题的解决进展。

6.与研发团队的结对开发

- 与研发人员保持紧密合作，了解正在开发的功能，及时编写相应的自动化测试脚本。
- 在功能开发过程中，参与设计和代码评审，确保开发过程中能够考虑到测试的需求。

- 通过结对开发的模式，确保开发和测试工作能够高效协同，减少后期的调试和问题修复时间。

6.5.3 工作流程

1.需求确认与功能开发同步

- 与研发团队合作，了解正在开发的功能和需求。
- 在开发初期就介入，确保测试用例设计与开发进度同步，提早识别潜在的测试问题。

2.测试脚本设计与编写

- 根据需求和开发进度，编写相应的自动化测试脚本。
- 对于难以自动化的测试场景，进行手动测试，并记录详细的测试结果。
- 将可自动化的测试项转化为自动化脚本，确保尽可能减少手动操作。

3.集成到 CI 流水线

- 配置并维护 Jenkins 等 CI 工具，确保每次开发提交后能够自动触发测试。
- 将自动化测试脚本集成到 Jenkins 中，并在每天的 DailyRun 中执行。
- 配置相关的通知系统（邮件群发形式），确保团队成员能够及时收到测试结果反馈。

4.测试执行与问题诊断

- 每天定时执行 DailyRun 测试，生成测试结果报告。
- 对于失败的测试项，立即进行分析，查找失败原因。
- 进行详细的调试并与研发人员共同排查问题，确保快速定位并解决问题。

5.日报生成与分享

- 每天根据测试结果，编写详细的测试日报，记录失败测试项及其 fail 原因。
- 在晨会上分享每日的测试结果，讨论失败的原因并提出改进措施。
- 对于重要的失败项，及时跟进问题的解决进度，并推动研发修复。

6.持续优化与迭代

- 根据 DailyRun 的测试结果，不断优化和调整自动化测试脚本，提升测试覆盖率和准确性。
- 对于发现的长期问题或频繁失败的测试项，进行专项优化或调整测试方案。

7.结对开发模式与协同工作

- 与研发人员保持紧密的合作，采用结对开发模式，共同评审需求和代码，确保测试在开发过程中的高效参与。
- 根据研发的反馈，调整测试策略和测试脚本，确保在每次功能开发完成后，测试能够快速验证。

6.6 运维管理工具开发、维护

6.6.1 职责范围

负责运维工具 NCenter、nRest、nBot 的开发、维护和审核工作，运维管理工具的职责范围主要包括：

1. 保障工具的正常运行：

- 维护软件各个功能的正常运行
- 工具提供给售后人员，需要适配售后人员遇到的公司生产的各种机型
- 面向测试部门内部，修复可能遇到的各种问题
- 由于 nBot 刷新工具提供给研发，打包进 FW 包中，需要保障刷新功能正常

2. 新功能开发：

- 满足客户对于运维软件的各种需求
- 针对测试部门内部同事使用场景，在运维软件中加入相应功能，实现工作效率的提升

6.6.2 工作内容

1. 运维工具的开发：

- NCenter、nRest、nBot 软件开发、编译与发版
- 软件和各个模块的升级包和安装包的开发

2. 用户手册的编写：

- 需要同步软件的发版更新用户手册

3. 工具发布前的审核

- 工具发布前需组内人员进行审核。通过多维度、全流程审核，确保功能完整性、运行稳定性、兼容性及安全性符合公司生产与用户使用要求

6.6.3 工作流程

1. 工具开发

- 了解软件开发需求
- 设计软件的整体架构，包括模块划分、数据流、接口设计等
- 进行工具的开发工作，满足需求

2. 功能测试

- 单元测试：确保每个模块的功能正确。
- 集成测试：测试模块之间的交互，确保整个系统按预期工作。
- 验收测试：邀请最终用户进行测试，收集反馈并进行调整。

3. 文档的编写

- 编写用户手册：提供详细的用户手册，帮助用户理解和使用软件。
- 编写开发文档：包括设计文档、代码注释、API 文档等，方便后续开发和维护

4. 内部审核

- 需求匹配度核查，逐一验证工具新增功能是否全部实现，无遗漏或功能缩水情况。
- 相关人员需要对代码审核，包括但不限于代码规范符合性审核、代码质量审核、代码可维护性审核。
- 验证用户手册是否与当前发布版本的功能完全匹配，新增功能需包含详细的操作步骤、界面说明、常见问题解答。
- 如果客户需要，则相关人员需要提供对应的测试报告。

5. 工具发版

- 验证软件功能是否正常
- 制作软件和模块安装包、升级包并上传至后台

6. 版本控制与代码管理

- 版本控制：使用 Git 进行版本控制，记录每次代码变更。
- 分支管理：使用分支管理策略（如 Git Flow）来管理开发和发布流程

7. 开发规范

代码风格：

- 遵循统一的代码风格规范，包括命名规范、缩进、注释等。
- 每个脚本的函数、变量名应尽可能简洁且富有描述性。

模块化与重用：

- 编写通用的函数和模块，避免重复代码。可重用的代码应提取到公共模块中，以便其他测试脚本调用。

异常处理：

- 处理所有可能的异常，确保测试脚本在执行过程中不会中断。
- 对于不可控的异常情况，应记录详细的日志信息，便于后续排查。

注释与文档：

- 每个自动化测试脚本和函数应有清晰的注释，说明功能和参数。
- 编写《脚本使用手册》和《脚本执行注意事项文档》，提供清晰的使用指南和注意事项。

版本控制与提交：

- 所有自动化测试脚本应按项目要求提交到 Git 仓库，并在提交前进行代码审查。
- 定期同步和更新 SVN 上的可执行文件，确保团队成员使用最新版本的脚本。

8. 交付物与输出

- **自动化测试脚本代码：**完整的自动化测试脚本代码，提交至 Git 版本管理系统和 svn 平台。
- **脚本执行报告：**自动化测试执行结果的报告，包含测试用例的执行结果、日志和截图。
- **《脚本使用手册》：**提供脚本执行的详细步骤、要求和注意事项。
- **《脚本执行相关培训资料》：**如果需要，编写相关培训材料，帮助团队成员掌握脚本的使用方法。
- **脚本执行注意事项文档：**总结脚本执行时的注意事项，确保脚本的正确性和高效性。

9. 常见问题和解决方案

9.1 脚本执行失败

- **原因：**环境问题（如依赖库缺失、环境变量未配置等）。

- **解决方案：**检查执行环境，确保所有依赖项已正确安装，并确保脚本执行的操作系统和平台配置正确。

9.2 自动化测试用例与手动用例无法对应

- **原因：**用例标签未正确更新或关联。
- **解决方案：**核对 DML 系统中的用例标签，并确保将相关手动测试用例更新为自动化测试执行标签。

9.3 脚本维护困难

- **原因：**代码缺乏清晰的注释和文档。
- **解决方案：**增强代码的可读性，确保代码有足够的注释，并定期进行脚本的复审和重构。

10.绩效考核标准

- **提高运维效率，进行运维工具的开发、维护、推广和使用：**保障重点客户运维工具按照客户要求期限送测使用，无特殊情况，每延迟 1 周扣 1 分
- **自动化覆盖率提升：**BIOS DML 系统全功能测试用例集自动化率大于 50%，BMC DML 系统全功能测试用例集自动化率大于 60%
- **重点 JDM 客户套件和稳定性测试开发：**按照客户和项目周期时间点完成自动化测试开发保证项目测试正常进行。

11.参考资料

- 《自动化测试框架设计与实现》
(https://wenku.baidu.com/view/4ad3d2cb28160b4e767f5acfa1c7aa00b42a9d42.html?fr=aladdin266&ind=1&aigcsid=0&qtype=0&lcid=1&queryKey=%E3%80%8A%E8%87%AA%E5%8A%A8%E5%8C%96%E6%B5%8B%E8%AF%95%E6%A1%86%E6%9E%B6%E8%AE%BE%E8%AE%A1%E4%B8%8E%E5%AE%9E%E7%8E%B0%E3%80%8B&verifyType=undefined&_wkts_=1758099847334&bdQuery=%E3%80%8A%E8%87%AA%E5%8A%A8%E5%8C%96%E6%B5%8B%E8%AF%95%E6%A1%86%E6%9E%B6%E8%AE%BE%E8%AE%A1%E4%B8%8E%E5%AE%9E%E7%8E%B0%E3%80%8B)
- PEP8 官方文档 (<https://pep8.org/>)
- Selenium 官方文档 (<https://www.selenium.dev/>)
- Git 官方文档 (<https://git-scm.com/docs>)

12.发行和管制

- 本文档的版本更新需由测试工具/脚本开发团队进行管理。
- 所有开发的脚本和相关文档需通过 Git 和 SVN 进行版本控制。